

Provue Engineering Take-Home — Pre-requisites Setup Checklist

This checklist gets your environment ready to build **Tara**, the finance-research persona described in the Provue Engineering Take-Home assignment. Work through it before you start the build — most of the 8–10 hour budget should go into schema, tools, agent, and orchestration, not into installing things.

Tick every item below before opening the assignment document.



Setup Checklist

✓ 1. Node.js and TypeScript

Install Node.js

- ☐ Install Node.js **v18 or newer** (LTS recommended) from <https://nodejs.org/>
- ☐ Verify in a terminal:

```
node --version npm --version
```

- ☐ You should see Node 18+ and npm 9+ printed

TypeScript runner

- ☐ You will run TypeScript scripts directly with `tsx`. Install it globally (optional but convenient):

```
npm install -g tsx typescript
```

- ☐ Verify: `tsx --version` and `tsc --version` both print a version

✓ 2. Mastra Project Setup

The assignment must be built using the **Mastra SDK**. Scaffold a fresh project to make sure the CLI works on your machine before the clock starts.

- ☐ Scaffold a sandbox project:

```
npm create mastra@latest
```

- ☐ During the wizard, select at minimum: **Agents** and **Tools** (Workflows is optional for this assignment)

- ☐ Pick a model provider you have a key for: **OpenAI**, **Anthropic**, or **Google**
- ☐ Boot the dev server:

```
cd <project> npm run dev
```

- ☐ Open the Mastra playground in the browser (the CLI prints the URL on startup) and confirm the sample agent responds
- ☐ Bookmark the docs you will actually need during the build:
 - ☐ [Quickstart](#)
 - ☐ `createTool` [reference](#)
 - ☐ [Agent reference](#)
 - ☐ [Adding Mastra to an Express server](#)
 - ☐ [Text-to-SQL template \(closest example\)](#)

✅ 3. Model Provider Account & API Key

You decide which provider to use. Pick one and make sure the key works *before* building anything.

- ☐ Account created with one of: [OpenAI](#), [Anthropic](#), or [Google AI Studio](#)
- ☐ Email + phone verification complete
- ☐ API key generated and stored in a password manager (never paste in chat, never commit)
- ☐ **Billing:** a payment method or pre-paid credits on file. Free tiers usually do not cover tool-calling models.
- ☐ Set a low monthly spend limit (e.g. \$10) for safety
- ☐ Sanity-check the key with a one-line script:

```
OPENAI_API_KEY=sk-... npx tsx -e "console.log('key length:', process.env.OPENAI_API_KEY?.length)"
```

- ☐ Add the key to a local `.env` file (which is in `.gitignore`)
- ☐ **If you cannot get a key:** mention it in your README — Provue will help unblock you

✅ 4. Postgres 14+

The assignment uses Postgres as the storage layer behind your tools. The shipped JSON is sample seed data only — your ingest script will populate Postgres, and your tools will query Postgres at request time.

Install Postgres

- ☐ Install **Postgres 14 or newer**. Pick one path:
 - ☐ **macOS (Homebrew):** `brew install postgresql@16` then `brew services start postgresql@16`

- ☐ **Ubuntu/Debian:** `sudo apt install postgresql-16`
- ☐ **Windows:** the installer from [postgresql.org](https://www.postgresql.org)
- ☐ **Docker (fastest):**

```
docker run -d --name provue-pg -p 5432:5432 -e POSTGRES_PASSWORD=postgres postgres:16
```
- ☐ Verify the server is up: `psql --version` prints a version, and `psql -U postgres -c 'select 1'` connects.

Create the assignment database

- ☐ Create a database for the assignment:

```
createdb provue_tara # or, via psql: # psql -U postgres -c "CREATE DATABASE provue_tara;"
```

- ☐ Save the connection string somewhere you'll remember (you'll need it in your `.env`):

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/provue_tara
```

- ☐ Confirm a client tool can connect: `psql` (CLI), [pgAdmin](#), [DBeaver](#), or the VS Code **SQLTools** extension.

Pick a query approach (no specific ORM required)

- ☐ Decide upfront how you'll talk to Postgres: **raw SQL** (`pg`), **Drizzle**, **Prisma**, **Knex**, or similar. Whatever you'll be productive in.
- ☐ Practice a tiny round-trip: `CREATE TABLE`, `INSERT`, `SELECT` from your chosen approach. Don't learn the library on assignment day.

Plan for managed Postgres at deploy time

- ☐ Pick a free-tier hosted Postgres for the deployed service: [Neon](#), [Supabase](#), [Render Postgres](#), or [Railway Postgres](#). Each gives you a `DATABASE_URL` in under five minutes.
- ☐ Verify the hosted DB accepts a connection from your laptop with the connection string they hand you.

✓ 5. GitHub Repository

- ☐ GitHub account ready (use a professional username — recruiters see it)
- ☐ Git installed and configured:

```
git config --global user.name "Your Name" git config --global user.email "you@example.com"
```

- ☐ SSH key added to GitHub or a personal access token configured for HTTPS
- ☐ Create an empty private repo for your submission (you can flip it public when you submit)
- ☐ Add a `.gitignore` that excludes `node_modules/`, `.env`, `dist/`, and any cached LLM logs

✓ 6. Deployment Platform

The finished service must be reachable at a public URL, with a working Postgres connection on the deployed side. Pick a host you already know — do not learn a new platform on assignment day.

- ☐ Account on one of: [Render](#), [Railway](#), [Fly.io](#), [Vercel](#) (paired with hosted Postgres), [Heroku](#), or your own AWS / GCP / Azure
- ☐ Free tier or trial credits cover both **a small Node service** and **a small Postgres database**. Render and Railway bundle both; Vercel needs a separate Postgres (Neon/Supabase).
- ☐ Deployment CLI installed if the platform uses one (`flyctl` , `railway`)
- ☐ You have run a "hello world" Node deploy on this platform at least once before
- ☐ You know how to set env vars on the platform — `DATABASE_URL` and your model API key both need to be configured for the deployed service

✓ 7. Local Dev Environment

- ☐ Editor of choice installed: [VS Code](#) or Cursor
- ☐ Recommended extensions:
 - ☐ TypeScript & JavaScript Language Features (built-in)
 - ☐ Prettier — code formatter
 - ☐ ESLint
 - ☐ GitLens
 - ☐ REST Client (for hitting `POST /ask` while developing)
 - ☐ SQLTools or PostgreSQL — for inspecting tables and running queries from the editor
- ☐ A terminal you are comfortable in (Terminal.app, iTerm2, Windows Terminal, etc.)
- ☐ A way to send `POST /ask` requests: `curl`, [Postman](#), or the REST Client extension

✓ 8. Sample Data and the Ingest Step

Three sample snapshots ship in the assignment's `data/` folder. They are **sample seed data only** — your implementation must load them into Postgres via an ingest script and query the database from your tools.

- ☐ You see three folders under `data/`: `sample_a/`, `sample_b/`, `sample_c/` — each contains `transactions.json`, `funds.json`, `holdings.json`
- ☐ You have opened all three and compared them: different merchants appear in different folders; one folder uses more NEFT-style memos. The variation is the in-band signal that **your tools must not hardcode merchant names, fund ids, aliases, or category lists**.
- ☐ You have spotted the deliberate complications: refunds (negative `amount`), merchant aliases (`Swiggy` / `Swiggy Instamart` / `SWIGGY*ORDER` / `SWIGGY BANGALORE`), self-transfers, missing categories (`uncategorized`), noisy UPI/NEFT memos.

- ☐ You understand the grading model: Provue runs your ingest script against a **fourth snapshot you have never seen**, then hits `POST /ask`.
- ☐ You have a plan for your ingest contract: e.g. `DATA_DIR=./data/sample_a npx tsx scripts/ingest.ts` — your script accepts a snapshot path and populates the database.

✓ 9. Concepts to Review

The assignment is testing engineering judgment, not framework recall. Make sure these are sharp before you start.

- ☐ **LLM tool calling**: how the model emits a tool call, how the runtime executes it, how the result is fed back
- ☐ **Mastra primitives**: `createTool`, `Agent`, the agent loop
- ☐ **Zod schemas**: validating tool inputs from a non-deterministic caller
- ☐ **Grounding**: why every number must come from a tool query, not the model's prose
- ☐ **Multi-step orchestration**: handling questions that need two or more tool calls (compare, rank, derive, join)
- ☐ **Express 5**: a single `POST /ask` route, JSON request and response
- ☐ **SQL fundamentals**: `WHERE`, `GROUP BY`, aggregates, dates, joins. Indexes on the columns your tools filter and group by.
- ☐ **Schema design**: what tables you'd create from `transactions.json` + `funds.json` + `holdings.json`, where foreign keys belong, what your `fund_nav` table looks like (one row per `(fund_id, date)` NAV point).
- ☐ **Fund vs holding return math**: period return (NAV change between two dates) vs the user's realised return on a specific holding (current value vs purchase cost) — these are different numbers.

✓ 10. Final Verification Before You Start

- ☐ Mastra sandbox project boots and the sample agent answers a question
- ☐ Model API key works from a script — non-zero token count returned
- ☐ **Local Postgres is running and accessible**; `provue_tara` database exists and accepts a connection from `psql` or a client
- ☐ You can deploy a "hello world" Node service to your chosen platform and hit it from your browser
- ☐ You have a hosted Postgres connection string ready for the deployed environment
- ☐ An empty submission repo is on GitHub with `.gitignore` and a stub `README.md`
- ☐ All three `data/sample_*/` snapshots open cleanly in your editor; you have eyeballed the variation between them
- ☐ You have read the assignment brief end-to-end **at least once** and noted the multi-snapshot grading model and the reliability rules

- ☐ You have skimmed `DESIGN.md` expectations — schema, indexes, formulas for spend, net spend, merchant matching, recurring detection, fund period return, holding realised return
 - ☐ You know which model provider, which deployment target (app + Postgres), and roughly how many tools and tables you intend to build
-

What You Will Submit

- ☐ A GitHub repo (or zip) containing your schema, ingest script, code, `README.md`, `DESIGN.md`, and an eval script
- ☐ A public URL where `POST /ask` returns `{ "answer": "..."}` for any question, backed by a deployed Postgres
- ☐ Observability evidence: logs, trace output, or screenshots showing one successful run and one handled no-data / failure run
- ☐ An eval script with at least 12 questions covering single lookup, date filtering, refunds, merchant aliases, transfers, category comparison, recurring subscriptions, no-data cases, fund period returns, and realised returns on holdings

Stuck?

If a setup step blocks you for more than 30 minutes, ask. Knowing when to ask a good question is one of the things the assignment is testing. Reach out before the clock starts — debugging Node or Postgres installs is not what the 8–10 hours are for.

Setup Checklist — for the Provue Engineering Take-Home: Build a Finance-Research Agent.